I. INTRODUCTION: THE POWER OF UNLABELED DATA

In Supervised Learning, we provide the AI with answers (labels). However, in the "Big Data" era, over 90% of data is unlabeled. **Unsupervised Learning** allows algorithms to explore data autonomously to find hidden patterns, group similar items, or compress information without human intervention.

1.1. Key Applications

- **Customer Segmentation:** Grouping millions of users based on purchasing behavior.
- **Anomaly Detection:** Identifying fraudulent transactions that do not fit the "normal" cluster.
- **Topic Modeling:** Automatically categorizing documents in a RAG system based on semantic similarity.

II. CLUSTERING ALGORITHMS: GROUPING THE UNKNOWN

2.1. K-Means Clustering

The most popular clustering algorithm. It partitions data into K distinct, non-overlapping subgroups (clusters).

- **The Process:** 1. Initialize K centroids randomly.

2. Assign each data point to the nearest centroid.

3. Re-calculate centroids based on the mean of the points in the cluster.

4. Repeat until centroids stop moving.

- **The Elbow Method:** We plot the **Inertia** (sum of squared distances) against K. The "elbow" of the curve indicates the optimal number of clusters where adding more clusters doesn't significantly improve the fit.

2.2. Hierarchical Clustering

Unlike K-Means, this creates a tree-like structure of clusters.

- **Dendrograms:** A visualization tool that shows the hierarchical relationship between clusters. It allows engineers to decide at what "level" to cut the tree to form groups.
- **Agglomerative:** A "bottom-up" approach where each point starts as its own cluster and merges with the closest neighbor.

III. DIMENSIONALITY REDUCTION: COMPRESSING BIG DATA

3.1. Principal Component Analysis (PCA)

Big Data often suffers from the **"Curse of Dimensionality"**—having too many features (columns) which leads to noise and slow computation. PCA is a mathematical technique that transforms high-dimensional data into a lower-dimensional form while retaining maximum **Variance**.

- **Principal Components:** New, uncorrelated variables that are linear combinations of the original features.

- **Explained Variance Ratio:** Tells us how much information (variance) is captured by each principal component. Usually, we aim for components that explain >95% of the total variance.

3.2. t-Distributed Stochastic Neighbor Embedding (t-SNE)

While PCA is linear, **t-SNE** is a non-linear technique specifically designed for **Data Visualization**. It is excellent at projecting high-dimensional word embeddings (from NLP) into 2D or 3D space to see how "similar" words cluster together.

IV. EVALUATING UNSUPERVISED MODELS

Since there are no labels, we cannot use "Accuracy." Instead, we use:

- **Silhouette Score:** Measures how similar an object is to its own cluster compared to other clusters. A score near +1 indicates highly dense and well-separated clusters.
- **Inertia:** Measures how tightly packed the clusters are internally.

V. PRACTICAL IMPLEMENTATION: CLUSTERING PIPELINE

Python

```
from sklearn.cluster import KMeans

from sklearn.decomposition import PCA

import matplotlib.pyplot as plt


# 1. Dimensionality Reduction to 2D for visualization

pca = PCA(n_components=2)

X_reduced = pca.fit_transform(X_large_dataset)


# 2. Finding the optimal K using the Elbow Method

inertia = []

for k in range(1, 11):

    kmeans = KMeans(n_clusters=k, n_init=10)

    kmeans.fit(X_reduced)

    inertia.append(kmeans.inertia_)


# 3. Final Clustering
```

```
final_model = KMeans(n_clusters=4)

labels = final_model.fit_predict(X_reduced)


# 4. Visualization

plt.scatter(X_reduced[:, 0], X_reduced[:, 1], c=labels, cmap='viridis')

plt.title("Cluster Visualization after PCA")

plt.show()
```